

```

# Mount Google Drive
from google.colab import drive
drive.mount('/content/drive')

# =====
# Environment Setup
# =====
print("Setting up environment...")

# Install required packages
!pip install -Uqq tensorflow tensorflow-datasets

Setting up environment...

import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow.keras import layers, models

print(f"TensorFlow version: {tf.__version__}")
print("Environment setup complete.\n")

# =====
# Configuration
# =====
print("Setting configuration parameters...")

# Dataset parameters
DATASET_NAME = 'tf_flowers'
NUM_CLASSES = 5
IMG_SIZE = (128, 128)
BATCH_SIZE = 16
BUFFER_SIZE = 1000

# Model parameters
BASE_MODEL = tf.keras.applications.MobileNetV2
FREEZE_LAYERS = True
INPUT_SHAPE = IMG_SIZE + (3,)

# Training parameters
INITIAL_EPOCHS = 20
PATIENCE = 3
VALIDATION_SPLIT = 0.2
SAVE_PATH = "/content/drive/MyDrive/flower_classifier.keras"

print(f"Using {BASE_MODEL.__name__} with image size {IMG_SIZE}")
print(f"Batch size: {BATCH_SIZE}, Epochs: {INITIAL_EPOCHS}")
print(f"Checkpoint path: {SAVE_PATH}\n")

# =====
# Data Preparation
# =====

```

```

print("Preparing dataset...")

def preprocess_image(image, label):
    # Resize and normalize images
    image = tf.image.resize(image, IMG_SIZE)
    image = tf.keras.applications.mobilenet_v2.preprocess_input(image)
    return image, label

def data_augment(image, label):
    # Using only native TF operations
    image = tf.image.random_flip_left_right(image)
    image = tf.image.random_brightness(image, max_delta=0.2)
    image = tf.image.random_contrast(image, lower=0.8, upper=1.2)
    image = tf.image.random_saturation(image, lower=0.8, upper=1.2)
    # Random 90-degree rotations
    if tf.random.uniform(()) > 0.5:
        image = tf.image.rot90(image)
    return image, label

# Load dataset
print("Loading dataset...")
(ds_train, ds_validation, ds_test), ds_info = tfds.load(
    name=DATASET_NAME,
    split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],
    as_supervised=True,
    with_info=True
)

# Create pipelines
print("Creating data pipelines...")
train_ds = ds_train.map(data_augment,
    num_parallel_calls=tf.data.AUTOTUNE)
train_ds = train_ds.map(preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE)
train_ds = train_ds.shuffle(BUFFER_SIZE).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

val_ds = ds_validation.map(preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE)
val_ds = val_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

test_ds = ds_test.map(preprocess_image,
    num_parallel_calls=tf.data.AUTOTUNE)
test_ds = test_ds.batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print(f"Training samples: {ds_info.splits['train'].num_examples * 0.8:.0f}")
print(f"Validation samples: {ds_info.splits['train'].num_examples * 0.1:.0f}")

```

```

print(f"Test samples: {ds_info.splits['train'].num_examples *
0.1:.0f}")
print("Data preparation complete.\n")

# =====
# Model Configuration
# =====
print("Building model...")

# Create base model
base_model = BASE_MODEL(
    input_shape=INPUT_SHAPE,
    include_top=False,
    weights='imagenet'
)

if FREEZE_LAYERS:
    base_model.trainable = False

# Build classification head
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(NUM_CLASSES, activation='softmax')
])

model.summary()
print("\nModel built successfully.\n")

# =====
# Training Setup
# =====
print("Configuring training...")

model.compile(
    optimizer=tf.keras.optimizers.Adam(),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=PATIENCE,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ModelCheckpoint(
        filepath=SAVE_PATH,
        save_best_only=True,
        monitor='val_accuracy'
    )
]

```

```

    )
]

print("Training configuration complete.\n")

# =====
# Training Execution
# =====
print("Starting training...")

history = model.fit(
    train_ds,
    epochs=INITIAL_EPOCHS,
    validation_data=val_ds,
    callbacks=callbacks
)

print("\nTraining complete.\n")

# =====
# Evaluation
# =====
print("Evaluating model...")

model = tf.keras.models.load_model(SAVE_PATH)
test_loss, test_acc = model.evaluate(test_ds)
print(f"\nTest accuracy: {test_acc:.4f}")
print(f"Test loss: {test_loss:.4f}")

# =====
# Resource Cleanup
# =====
print("\nCleaning up resources...")

del ds_train, ds_validation, ds_test
del train_ds, val_ds, test_ds
tf.keras.backend.clear_session()

print("Cleanup complete.")

TensorFlow version: 2.19.0
Environment setup complete.

Setting configuration parameters...
Using MobileNetV2 with image size (128, 128)
Batch size: 16, Epochs: 20
Checkpoint path: /content/drive/MyDrive/flower_classifier.keras

Preparing dataset...
Loading dataset...
Creating data pipelines...

```

Training samples: 2936
Validation samples: 367
Test samples: 367
Data preparation complete.

Building model...

Model: "sequential_2"

Layer (type) Param #	Output Shape
mobilenetv2_1.00_128 (Functional)	(None, 4, 4, 1280)
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 1280)
dense_2 (Dense)	(None, 5)

Total params: 2,264,389 (8.64 MB)

Trainable params: 6,405 (25.02 KB)

Non-trainable params: 2,257,984 (8.61 MB)

Model built successfully.

Configuring training...
Training configuration complete.

Starting training...

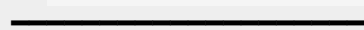
Epoch 1/20

184/184 35s 120ms/step - accuracy: 0.6214 - loss: 0.9922 - val_accuracy: 0.8365 - val_loss: 0.4186

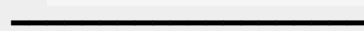
Epoch 2/20

184/184 22s 44ms/step - accuracy: 0.8528 - loss: 0.4142 - val_accuracy: 0.8638 - val_loss: 0.3694

Epoch 3/20

184/184  12s 49ms/step - accuracy: 0.8765 - loss: 0.3463 - val_accuracy: 0.8801 - val_loss: 0.3247

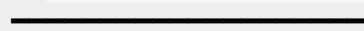
Epoch 4/20

184/184  20s 43ms/step - accuracy: 0.8820 - loss: 0.3043 - val_accuracy: 0.9046 - val_loss: 0.3207

Epoch 5/20

184/184  21s 40ms/step - accuracy: 0.9038 - loss: 0.2707 - val_accuracy: 0.8719 - val_loss: 0.3366

Epoch 6/20

184/184  12s 47ms/step - accuracy: 0.9297 - loss: 0.2271 - val_accuracy: 0.8883 - val_loss: 0.3230

Epoch 7/20

184/184  12s 51ms/step - accuracy: 0.9269 - loss: 0.2173 - val_accuracy: 0.9019 - val_loss: 0.3259

Training complete.

Evaluating model...

23/23  7s 139ms/step - accuracy: 0.8572 - loss: 0.4324

Test accuracy: 0.8856

Test loss: 0.3708

Cleaning up resources...

Cleanup complete.